

Utiliser impérativement les feuilles quadrillées fournies pour les schémas

L'objectif de cet exercice est de comparer la performance d'exécution d'un algorithme de tri sur deux réalisations différentes du Mips. La première est la réalisation classique sur un pipeline de 5 étages présenté en cours. Cette réalisation est appelée **Mips**. La seconde, appelée **SS2**, est une réalisation superscalaire à deux pipelines. Il s'agit de la réalisation à 5 étages présentée en cours. **SS2** à la même fréquence d'horloge que la réalisation **Mips**.

Un des algorithmes de tri les plus performants est le **QuickSort** (tri rapide). Cet algorithme est appliqué à un tableau d'entiers. Le cœur de cet algorithme consiste à trouver la place définitive du premier élément du tableau (appelé pivot), dans le tableau trié, de le placer. Les éléments plus grands que le pivot sont placés après celui-ci et les éléments plus petits avant.

```
unsigned int PlacerPivot (int *tab, unsigned int size)
{
    unsigned int idx    = 1;
    int          pivot  = 0;
    int          tmp    = 0;
    unsigned int i      = 1;

    pivot = tab [0];
    for (i=1 ; i!=size ; i++)
    {
        if (tab [i] < pivot)
        {
            tmp      = tab [i ] ;
            tab [idx] = tmp      ;
            tab [i ]  = tab [idx];
            idx ++;
        }
    }
    tab [0 ] = tab [idx-1];
    tab [idx-1] = pivot      ;

    return (idx);
}
```

On dispose de deux versions pour le code de la boucle principale de cette fonction en assembleur Mips pipeline. A l'entrée de la boucle, le registre R4 contient l'adresse de l'élément 1 du tableau (l'adresse de **tab [1]**). Le registre R5 contient l'adresse de fin du tableau, le registre R8 la valeur du pivot et le registre R9, l'adresse de l'élément d'index **idx**.

1ère version :

```
Loop1 :      Lw      r10, 0 (r4 )          ; tab [i]
             Slt     r11, r10, r8          ; tab [i] < pivot
             Beq     r11, r0 , Endif1
             Nop
             Lw      r12, 0 (r9 )          ; tab [idx]
             Sw      r12, 0 (r4 )
             Sw      r10, 0 (r9 )
             Addiu   r9 , r9 , 4

Endif1 :     Addiu   r4 , r4 , 4
             Bne     r4 , r5 , Loop1
             Nop
```

2ème version :

```
Loop2 :      Lw      r10, 0 (r4 )      ; tab [i ]
             Lw      r12, 0 (r9 )      ; tab [idx]
             Sw      r12, 0 (r4 )
             Sw      r10, 0 (r9 )
             Slt     r11, r10, r8      ; tab [i] < pivot
             Sll     r11, r11, 2
             Addu    r9 , r9 , r11
             Addiu   r4 , r4 , 4
             Bne     r4 , r5 , Loop2
             Nop
```

- 1- Analyser l'exécution de la boucle **Loop1** sur la réalisation **Mips** dans le cas où le branchement **Beq** réussit et dans le cas où il échoue. Il n'est pas demandé de faire un schéma mais, d'indiquer simplement les dépendances de données et les situations qui provoquent les cycles de gel. Calculer le nombre moyen de cycles par itération en supposant que dans 50% des cas le branchement réussit. Calculer le CPI et le CPI-utile.
- 2- Analyser l'exécution de la boucle **Loop1** à l'aide d'un schéma simplifié sur la réalisation **SS2** dans le cas où le branchement **Beq** réussit et dans le cas où il échoue. On suppose que l'étiquette **Loop1** se trouve sur une adresse **alignée** et que le buffer d'instructions est vide au début de l'exécution de la boucle. Calculer le nombre moyen de cycles par itération en supposant que dans 50% des cas le branchement réussit. Calculer le CPI et le CPI-utile.
- 3- Analyser l'exécution de la boucle **Loop2** sur la réalisation **Mips**. Il n'est pas demandé de faire un schéma mais, d'indiquer simplement les dépendances de données et les situations qui provoquent les cycles de gel. Calculer le nombre moyen de cycles par itération. Calculer le CPI et le CPI-utile.
- 4- Analyser l'exécution de la boucle **Loop2** à l'aide d'un schéma simplifié sur la réalisation **SS2**. On suppose que l'étiquette **Loop2** se trouve sur une adresse **alignée** et que le buffer d'instructions est vide au début de l'exécution de la boucle. Calculer le nombre moyen de cycles par itération. Calculer le CPI et le CPI-utile.
- 5- Pour la réalisation **Mips**, réordonner le code de **Loop1** afin d'éviter au maximum les cycles perdus. Calculer le nombre moyen de cycles par itération. Calculer le CPI et le CPI-utile dans la même hypothèse (50% des branchements réussissent).
- 6- Pour la réalisation **Mips**, réordonner le code de **Loop2** afin d'éviter au maximum les cycles perdus. Calculer le nombre moyen de cycles par itération. Calculer le CPI et le CPI-utile.
- 7- Pour la réalisation **Mips**, optimiser le code de **Loop1** à l'aide de la technique "pipeline logiciel" en indiquant clairement quelles instructions font partie de quel étage. Puis réordonner afin d'éviter au maximum les cycles perdus. Calculer le nombre moyen de cycles par itération. Calculer le CPI et le CPI-utile.
- 8- Pour la réalisation **SS2**, dérouler le code de **Loop2** une fois (traitement de deux éléments par itération) et optimiser le code de manière à éviter les cycles perdus au maximum.

Dans la suite, on considère le code original des deux versions. On s'intéresse à la réalisation **Mips**.

Le processeur est relié à la mémoire à travers un cache de données et un cache d'instructions. On souhaite étudier l'influence du cache de données sur la performance d'exécution. On suppose que le cache d'instructions est parfait (taux de Miss = 0).

Le cache de données est un cache *Write Through* à correspondance directe. Il a une capacité de 16 Koctets. Un bloc du cache contient 32 octets.

En cas de *Hit*, il faut 1 cycle pour accéder au cache.

En cas de *Miss*, il faut 1 cycle pour détecter le *Miss*. Puis, il faut en moyenne 3 cycles pour initialiser l'accès à la mémoire. Une fois l'accès mémoire initialisé, on peut transférer un mot à chaque cycle. Lorsque le bloc a été entièrement reçu, il faut 1 cycle pour mettre à jour les mémoires du cache et 1 cycle supplémentaire pour répondre au processeur.

Dans un premier temps, on ne s'intéresse qu'aux lectures en négligeant l'effet des écritures.

On suppose qu'au début de la boucle le cache est vide. Le tableau à trier contient 1 K entiers et l'adresse du tableau est alignée (multiple de 4096).

- 9- De combien de mots est constitué un bloc de cache. En cas de *Miss* combien de cycles dure une lecture.
- 10- Pour chacune des deux versions, combien de *Miss* se produisent-ils lors de l'exécution des itérations.
- 11- Calculer le CPI-utile de la boucle pour chacune des deux versions.
- 12- Comment évolue le taux de *Miss* si l'on remplace le cache *Write Through* à correspondance directe par un cache *Write Back* à correspondance directe. Justifier votre réponse.
- 13- Comment évolue le taux de *Miss* si l'on remplace le cache *Write Through* à correspondance directe par un cache *Write Through* associatif et si on augmente le niveau d'associativité. Justifier votre réponse.

On s'intéresse maintenant à l'effet des écritures.

Le cache de données *Write Through* à correspondance directe contient un buffer d'écritures postées d'une place. Du point de vue du processeur, l'écriture ne peut se faire que s'il y a une place dans le buffer. Dans ce cas, l'écriture dans le buffer nécessite 1 cycle. Lorsque le buffer contient une demande d'écriture, le cache met en moyenne 4 cycles pour effectuer l'écriture en mémoire et vider le buffer.

- 14- Pour chacune des deux versions, calculer le CPI-utile de la boucle en tenant compte de l'effet des lectures et des écritures.
- 15- En conservant un cache *Write Through*, quelles modifications faut-il apporter au cache pour réduire l'impact des écritures.
- 16- Calculer le CPI-utile si on met en œuvre la solution que vous proposez.